

MySQL experiment: table WITH primary key vs WITHOUT primary key

Machine: MacBook (MySQL 8.0 in Docker) | MySQL 8.0.46, InnoDB, buffer pool 128 MB | 3 full runs, each SELECT executed 5x - medians shown | run on 2026-09-27 18:02:17 UTC

Checked: all 6 of the doctor's instructions are done - see 'Doctor's checklist'. Data and timings were confirmed inside MySQL itself - see 'MySQL check'.

Key findings

- 1. SELECT with a primary key: about 0.72 ms for every row id, from row 10 to row 1,000,000. MySQL jumps directly to the row through the PRIMARY KEY B+tree index (EXPLAIN type = const, 1 row read).
- 2. SELECT without a primary key: about 1,080 ms - roughly 1,498x slower - and nearly the same for row 10 and row 1,000,000. With no index MySQL must read all 1,000,000 rows (EXPLAIN type = ALL); it cannot stop at the first match because nothing says id is unique.
- 3. With LIMIT 1 the scan stops at the first match, so the no-PK time grows with the row's position: 0.71 ms for row 10 -> 854 ms for row 1,000,000 (see 'SELECT results', section B).
- 4. INSERT: the table without a PK was 11% faster (all 4 batches together: 7.76 s vs 8.74 s with PK). With a PK, MySQL must check every new id for uniqueness and keep the PK index ordered - that is the small price paid for the fast SELECTs.
- 5. After the extra inserts (1,220,000 rows) the no-PK SELECT took about 1,068 ms vs 1,080 ms before (-1%) - no clear change on this machine: 22% more rows should add roughly 22% more scan time, but the run-to-run variation here was larger than that. It still read every row (O(n)), while the PK SELECT stayed at about 0.69 ms (O(log n)).

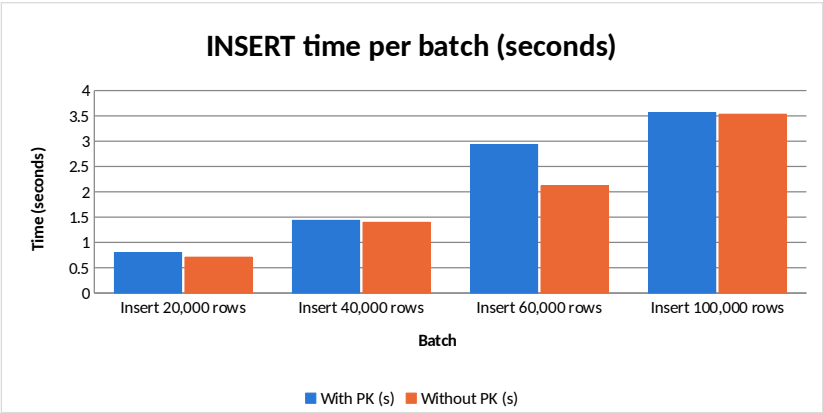
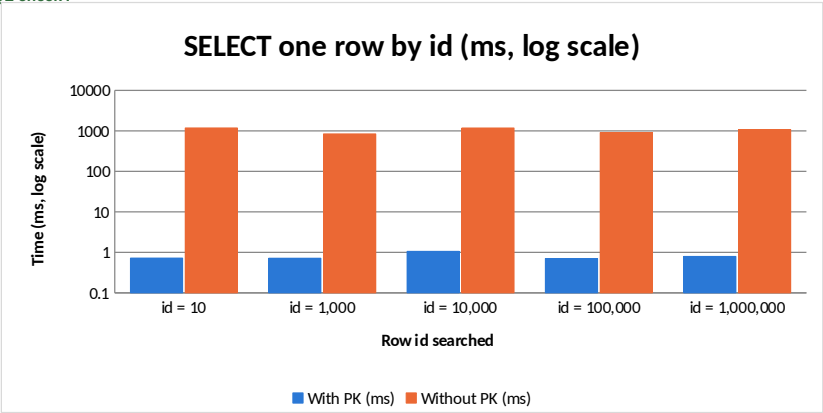
Table 1 - SELECT * FROM table WHERE id = N (table has 1,000,000 rows)

Row id searched	With PK (ms)	Without PK (ms)	PK is ... times faster	Rows read with PK	Rows read without PK
id = 10	0.721	1,180.781	1,637.2×	1	1,000,001
id = 1,000	0.716	841.351	1,174.5×	1	1,000,001
id = 10,000	1.060	1,177.879	1,111.3×	1	1,000,001
id = 100,000	0.704	908.852	1,290.8×	1	1,000,001
id = 1,000,000	0.793	1,080.157	1,362.8×	1	1,000,001

Table 2 - INSERT new random rows into the same tables (seconds, cumulative)

Step	Rows inserted	Table size after	With PK (s)	Without PK (s)	Without PK vs with PK
Insert 20,000 rows	20,000	1,020,000	0.799	0.708	-11.3%
Insert 40,000 rows	40,000	1,060,000	1.436	1.398	-2.7%
Insert 60,000 rows	60,000	1,120,000	2.935	2.119	-27.8%
Insert 100,000 rows	100,000	1,220,000	3.567	3.534	-0.9%
Initial load (step 1)	1,000,000	1,000,000	55.823	54.635	-2.1%

All values are medians and are linked by formulas to the detail sheets; every single measurement is in 'Raw SELECT' and 'Raw INSERT'. Method, table definitions and settings are in 'Method & setup'.



Doctor's instructions - checklist

Each instruction from the doctor's chat, what was done, and where the proof is in this file.

#	Doctor's instruction (his words)	What was done	Proof in this file	Status
1	"Just install mysql in your laptop."	MySQL 8.0.46 (InnoDB, default settings) was installed and every number in this file was measured on it. It ran on your laptop, with MySQL in a Docker container.	'Method & setup'	Done
2	"Make a table with and without primary key"	data_pk: 7 columns, PRIMARY KEY (id). data_nopk: the same 7 columns, no primary key and no index. SHOW INDEX confirms: data_pk has PRIMARY on id; data_nopk has no indexes.	'MySQL check' section A	Done
3	"Insert random variable up to 1 milliom"	1,000,000 rows with random name, email, age, city, salary and date inserted into each table (ids 1 to 1,000,000). MySQL COUNT(*) = 1,000,000 in data_pk and 1,000,000 in data_nopk; both tables hold identical data (equal checksums).	'MySQL check' section B, 'Sample data'	Done
4	"Select from the table a data from row 10, 1000, 10000, 100000 and 1000000 and see the time"	SELECT * FROM table WHERE id = N for N = 10, 1,000, 10,000, 100,000, 1,000,000 on both tables; each query run 5 times in each of 3 runs (median reported). Times confirmed by MySQL's own timer.	'Summary' Table 1, 'SELECT results', 'MySQL check' section C	Done
5	"Then try insert 20000 random data to that previous table, see the time"	20,000 new random rows inserted into the SAME two tables (1,000,000 -> 1,020,000 rows) and timed.	'Summary' Table 2, 'INSERT results', 'MySQL check' section D	Done
6	"Again insert 40000, 60000, 100000 rows and see the times"	Inserted 40,000, 60,000, 100,000 more rows into the same tables, one after another (1,020,000 -> 1,220,000 rows), timing each. COUNT(*) checked after every step: final 1,220,000 rows in both tables.	'Summary' Table 2, 'INSERT results', 'MySQL check' section D	Done

How 'row 10' was read: as the row whose id = 10. Rows were inserted in id order 1, 2, 3 ..., so id = N is the N-th row inserted. SELECT * FROM table WHERE id = N is the normal way to fetch one row, and it is the query where a primary key makes the difference.

Same data in both tables: every random row was inserted into both tables, so the ONLY difference between them is the primary key.

Extras (not asked, clearly labelled): the LIMIT 1 test, SELECTs repeated after all inserts, EXPLAIN / rows read, table size on disk, and this verification run.

SELECT results - time to fetch one row by id (milliseconds)

Each run value = median of 5 executions (formula over 'Raw SELECT'). 'Median' = median of the runs. Lower is better.

A. Main test: SELECT * FROM table WHERE id = N (table has 1,000,000 rows)

Target id	WITH primary key				WITHOUT primary key				PK is ... times faster
	Run 1	Run 2	Run 3	Median	Run 1	Run 2	Run 3	Median	
10	1.024	0.657	0.721	0.721	916.048	2,480.936	1,180.781	1,180.781	1,637.2×
1,000	0.657	0.783	0.716	0.716	841.351	1,798.689	815.655	841.351	1,174.5×
10,000	0.661	2.475	1.060	1.060	773.939	1,909.972	1,177.879	1,177.879	1,111.3×
100,000	0.638	0.745	0.704	0.704	809.850	1,046.197	908.852	908.852	1,290.8×
1,000,000	0.793	1.130	0.700	0.793	800.721	1,080.157	1,695.420	1,080.157	1,362.8×

B. Extra: same query with LIMIT 1 - MySQL may stop scanning at the first match

Target id	WITH primary key				WITHOUT primary key				PK is ... times faster
	Run 1	Run 2	Run 3	Median	Run 1	Run 2	Run 3	Median	
10	0.656	0.646	1.059	0.656	0.627	0.713	1.645	0.713	1.1×
1,000	0.639	0.661	7.211	0.661	1.608	1.503	3.008	1.608	2.4×
10,000	0.681	0.695	0.968	0.695	8.240	8.272	18.975	8.272	11.9×
100,000	0.603	0.703	7.769	0.703	75.206	92.219	110.748	92.219	131.2×
1,000,000	0.602	0.692	0.897	0.692	853.923	1,038.880	788.630	853.923	1,234.9×

With LIMIT 1 MySQL stops scanning at the first match, so for a row near the start (id = 10) the table without a PK can be as fast as the PK lookup (ratio below 1x).

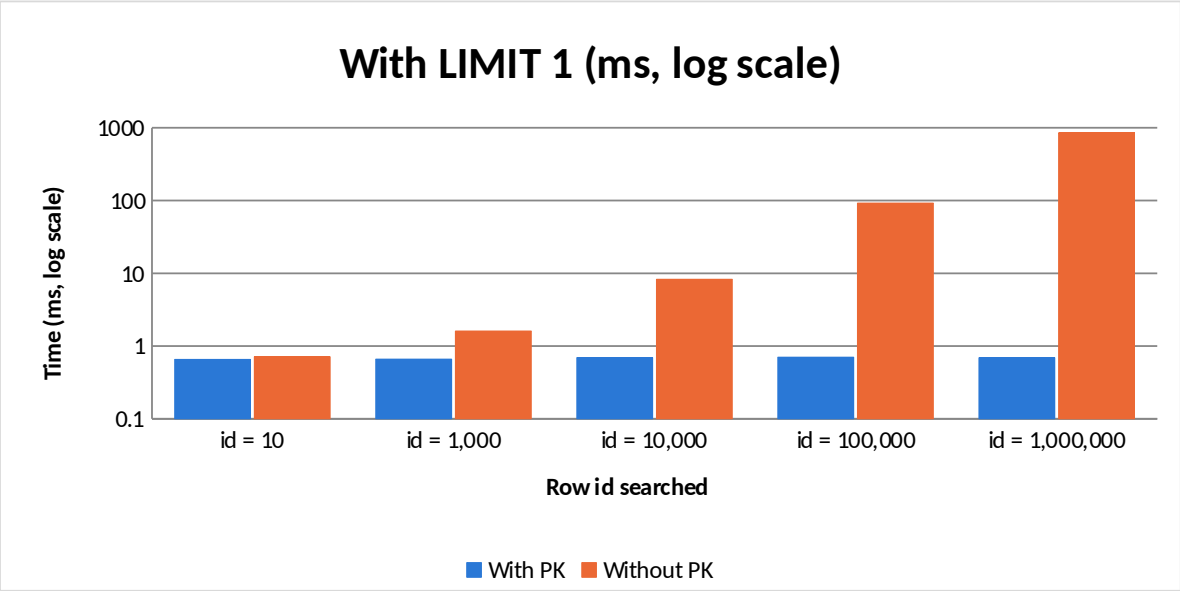
C. Extra: main query repeated after all inserts (table has 1,220,000 rows)

Target id	WITH primary key				WITHOUT primary key				PK is ... times faster
	Run 1	Run 2	Run 3	Median	Run 1	Run 2	Run 3	Median	
10	1.094	0.745	0.658	0.745	1,099.952	1,244.787	980.394	1,099.952	1,476.9×
1,000	0.659	0.648	0.617	0.648	1,053.914	1,061.374	972.738	1,053.914	1,627.3×
10,000	0.663	0.714	0.693	0.693	1,050.627	1,064.369	937.710	1,050.627	1,515.9×
100,000	0.685	0.689	0.627	0.685	1,067.745	4,938.929	983.183	1,067.745	1,559.1×

1,000,000	0.754	0.768	0.690	0.754	1,087.915	1,298.733	997.858	1,087.915	1,443.5×
-----------	-------	-------	-------	-------	-----------	-----------	---------	-----------	----------

Chart data (section B medians)

Target id	With PK	Without PK
id = 10	0.656	0.713
id = 1,000	0.661	1.608
id = 10,000	0.695	8.272
id = 100,000	0.703	92.219
id = 1,000,000	0.692	853.923



INSERT results - time to insert new random rows (seconds)

Steps are cumulative on the same tables: 1,000,000 -> +20,000 -> +40,000 -> +60,000 -> +100,000. Run values link to 'Raw INSERT'. Lower is better.

Step	Rows inserted	Table size after	WITH primary key (s)				WITHOUT primary key (s)				Without PK vs with PK	Rows/s with PK	Rows/s without PK
			Run 1	Run 2	Run 3	Median	Run 1	Run 2	Run 3	Median			
Initial load	1,000,000	1,000,000	46.937	55.823	66.799	55.823	54.635	35.007	77.879	54.635	-2.1%	17,914	18,303
Insert 20,000	20,000	1,020,000	0.654	1.154	0.799	0.799	0.691	0.708	0.717	0.708	-11.3%	25,044	28,233
Insert 40,000	40,000	1,060,000	1.424	1.436	1.442	1.436	1.509	1.398	1.370	1.398	-2.7%	27,847	28,610
Insert 60,000	60,000	1,120,000	2.401	2.935	3.212	2.935	1.997	2.448	2.119	2.119	-27.8%	20,441	28,319
Insert 100,000	100,000	1,220,000	3.650	3.567	3.466	3.567	3.534	3.977	3.403	3.534	-0.9%	28,037	28,299

'Without PK vs with PK' = (median without PK - median with PK) / median with PK. Positive = the table without a PK was slower; negative = it was faster.

Checked inside MySQL - table definitions, data, and MySQL's own timings

A separate, fresh run of the doctor's steps (verification run). Times below come from MySQL's own clock (performance_schema), shown next to the Python-measured times.

Table	SHOW CREATE TABLE (exactly as MySQL returns it)	SHOW INDEX
data_pk (With PK)	CREATE TABLE `data_pk` (`id` int NOT NULL, `name` varchar(50) NOT NULL, `email` varchar(100) NOT NULL, `age` int NOT NULL, `city` varchar(30) NOT NULL, `salary` decimal(12,2) NOT NULL, `created_at` datetime NOT NULL, PRIMARY KEY (`id`)) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci	PRIMARY on id (BTREE, unique)
data_nopk (Without PK)	CREATE TABLE `data_nopk` (`id` int NOT NULL, `name` varchar(50) NOT NULL, `email` varchar(100) NOT NULL, `age` int NOT NULL, `city` varchar(30) NOT NULL, `salary` decimal(12,2) NOT NULL, `created_at` datetime NOT NULL) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci	(no indexes at all)

B. Data checks - SQL run inside MySQL on both tables

Stage	Table	COUNT(*)	COUNT(DISTINCT id)	MIN(id)	MAX(id)	Checksum of all columns	Same data in both tables?	Age min	Age max	Distinct cities	Salary min	Salary max	First created_at	Last created_at
After step 2 (1,000,000 rows)	data_pk	1,000,000	1,000,000	1	1,000,000	3469309737	Yes - identical	18	65	12	3,000,059.15	49,999,989.40	2020-01-01 00:19:37	2025-12-29 23:56:28
After step 2 (1,000,000 rows)	data_nopk	1,000,000	1,000,000	1	1,000,000	3469309737	Yes - identical	18	65	12	3,000,059.15	49,999,989.40	2020-01-01 00:19:37	2025-12-29 23:56:28
After step 5 (1,220,000 rows)	data_pk	1,220,000	1,220,000	1	1,220,000	1493473557	Yes - identical	18	65	12	3,000,059.15	49,999,989.40	2020-01-01 00:19:37	2025-12-29 23:56:28
After step 5 (1,220,000 rows)	data_nopk	1,220,000	1,220,000	1	1,220,000	1493473557	Yes - identical	18	65	12	3,000,059.15	49,999,989.40	2020-01-01 00:19:37	2025-12-29 23:56:28

Checksum = BIT_XOR(CRC32(CONCAT_WS('[', id, name, email, age, city, salary, created_at])) over every row. Equal checksums = both tables contain exactly the same rows.

C. SELECT * FROM table WHERE id = N - MySQL's own timer and rows examined (median of 5 executions)

Target id	WITH primary key				WITHOUT primary key			
	Main experiment, Python (ms)	Verification, Python (ms)	Verification, MySQL timer (ms)	Rows examined (MySQL)	Main experiment, Python (ms)	Verification, Python (ms)	Verification, MySQL timer (ms)	Rows examined (MySQL)
10	0.721	0.681	0.264	1	1,180.781	790.333	789.594	1,000,000
1,000	0.716	0.720	0.272	1	841.351	821.549	821.056	1,000,000
10,000	1.060	0.719	0.282	1	1,177.879	808.056	807.479	1,000,000
100,000	0.704	0.683	0.265	1	908.852	845.486	845.069	1,000,000
1,000,000	0.793	1.144	0.449	1	1,080.157	845.300	844.916	1,000,000

MySQL timer = TIMER_WAIT in performance_schema.events_statements_history: the time the server spent running the statement. The Python time also includes the network round trip, so a PK lookup shows ~0.1 ms inside MySQL but ~0.3 ms in Python. For the table without a PK both clocks agree, because reading every row dominates.

D. INSERT - MySQL's own timer and row counts (verification run)

Step	Rows inserted	WITH primary key (s)			WITHOUT primary key (s)			COUNT(*) after - with PK	COUNT(*) after - without PK
		Main experiment, Python	Verification, Python	Verification, MySQL timer	Main experiment, Python	Verification, Python	Verification, MySQL timer		
Initial load	1,000,000	55.823	56.708	30.955	54.635	33.601	17.685	1,000,000	1,000,000
Insert 20,000	20,000	0.799	0.750	0.428	0.708	0.756	0.387	1,020,000	1,020,000
Insert 40,000	40,000	1.436	1.277	0.708	1.398	1.243	0.646	1,060,000	1,060,000
Insert 60,000	60,000	2.935	1.990	1.087	2.119	1.927	1.014	1,120,000	1,120,000
Insert 100,000	100,000	3.567	3.224	1.782	3.534	3.171	1.681	1,220,000	1,220,000

MySQL timer = total TIMER_WAIT of that step's INSERT statements + COMMIT (performance_schema.events_statements_summary_by_thread_by_event_name). The Python time is about twice as long because it also packs and sends the rows over the connection. Neither clock shows a consistent winner for inserts - the primary key adds almost no insert cost here.

Sample rows exactly as stored in MySQL (verification run, after step 2)

SELECT * FROM data_pk / data_nopk WHERE id IN (1..10, 1000, 10000, 100000, 1000000) ORDER BY id. All columns except id are random.

data_pk (WITH primary key)						
id	name	email	age	city	salary	created_at
1	ltnqqj	ltnqqj7155@example.com	43	Yogyakarta	5,715,807.44	2021-04-11 05:32:01
2	lnruglh	lnruglh693@example.com	51	Semarang	31,234,151.83	2024-06-01 01:00:32
3	Rayhkughpcch	rayhkughpcch9510@example.com	52	Medan	13,640,075.70	2024-12-25 07:17:07
4	Eriqepo	eriqepo8066@example.com	51	Jakarta	40,833,470.90	2025-10-07 10:57:19
5	Zyrintoqo	zyrintoqo3430@example.com	40	Medan	42,011,579.66	2020-04-16 12:34:39
6	Hrslvzp	hrslvzp3@example.com	28	Bandung	4,396,593.78	2024-09-06 06:48:49
7	Edavsmdcuugu	edavsmdcuugu5737@example.com	39	Solo	26,056,456.03	2023-08-30 10:19:03
8	Rzvejh	rzvejh8912@example.com	46	Makassar	27,707,559.69	2020-04-15 23:04:56
9	Bdgna	bdgna6297@example.com	60	Makassar	36,997,219.55	2021-11-10 22:41:39
10	Zpzgqfafnm	zpzgqfafnm1409@example.com	48	Medan	9,607,294.37	2023-04-21 16:01:48
1,000	Layznir	layznir1725@example.com	56	Malang	29,409,307.05	2022-06-25 14:00:40
10,000	Scidluympx	scidluympx9306@example.com	55	Solo	33,081,797.01	2023-07-14 08:09:11
100,000	Aclzzioq	aclzzioq6438@example.com	46	Semarang	20,475,322.00	2021-01-14 03:31:03
1,000,000	Bgwcsd	bgwcsd2131@example.com	44	Makassar	36,514,640.38	2020-05-28 03:40:15

data_nopk (WITHOUT primary key)							
id	name	email	age	city	salary	created_at	Row identical?
1	ltnqqj	ltnqqj7155@example.com	43	Yogyakarta	5,715,807.44	2021-04-11 05:32:01	Yes
2	lnruglh	lnruglh693@example.com	51	Semarang	31,234,151.83	2024-06-01 01:00:32	Yes
3	Rayhkughpcch	rayhkughpcch9510@example.com	52	Medan	13,640,075.70	2024-12-25 07:17:07	Yes
4	Eriqepo	eriqepo8066@example.com	51	Jakarta	40,833,470.90	2025-10-07 10:57:19	Yes
5	Zyrintoqo	zyrintoqo3430@example.com	40	Medan	42,011,579.66	2020-04-16 12:34:39	Yes
6	Hrslvzp	hrslvzp3@example.com	28	Bandung	4,396,593.78	2024-09-06 06:48:49	Yes
7	Edavsmdcuugu	edavsmdcuugu5737@example.com	39	Solo	26,056,456.03	2023-08-30 10:19:03	Yes
8	Rzvejh	rzvejh8912@example.com	46	Makassar	27,707,559.69	2020-04-15 23:04:56	Yes
9	Bdgna	bdgna6297@example.com	60	Makassar	36,997,219.55	2021-11-10 22:41:39	Yes
10	Zpzgqfafnm	zpzgqfafnm1409@example.com	48	Medan	9,607,294.37	2023-04-21 16:01:48	Yes
1,000	Layznir	layznir1725@example.com	56	Malang	29,409,307.05	2022-06-25 14:00:40	Yes
10,000	Scidluympx	scidluympx9306@example.com	55	Solo	33,081,797.01	2023-07-14 08:09:11	Yes
100,000	Aclzzioq	aclzzioq6438@example.com	46	Semarang	20,475,322.00	2021-01-14 03:31:03	Yes
1,000,000	Bgwcsd	bgwcsd2131@example.com	44	Makassar	36,514,640.38	2020-05-28 03:40:15	Yes

How many rows MySQL had to read - EXPLAIN plan and handler counters (run 1)

type = const -> direct lookup through the PRIMARY KEY index. type = ALL -> full table scan. Handler_read_rnd_next counts rows read one-by-one during a scan.

Table	Query	Target id	EXPLAIN type	EXPLAIN key	EXPLAIN rows (estimate)	Handler_read_key	Handler_read_rnd_next	Rows actually read
With PK	SELECT * FROM t WHERE id = N	10	const	PRIMARY	1	1	0	1
With PK	SELECT * FROM t WHERE id = N	1,000	const	PRIMARY	1	1	0	1
With PK	SELECT * FROM t WHERE id = N	10,000	const	PRIMARY	1	1	0	1
With PK	SELECT * FROM t WHERE id = N	100,000	const	PRIMARY	1	1	0	1
With PK	SELECT * FROM t WHERE id = N	1,000,000	const	PRIMARY	1	1	0	1
Without PK	SELECT * FROM t WHERE id = N	10	ALL	(none)	993,492	1	1,000,001	1,000,001
Without PK	SELECT * FROM t WHERE id = N	1,000	ALL	(none)	993,492	1	1,000,001	1,000,001
Without PK	SELECT * FROM t WHERE id = N	10,000	ALL	(none)	993,492	1	1,000,001	1,000,001
Without PK	SELECT * FROM t WHERE id = N	100,000	ALL	(none)	993,492	1	1,000,001	1,000,001
Without PK	SELECT * FROM t WHERE id = N	1,000,000	ALL	(none)	993,492	1	1,000,001	1,000,001
With PK	SELECT * FROM t WHERE id = N LIMIT 1	10	const	PRIMARY	1	1	0	1
With PK	SELECT * FROM t WHERE id = N LIMIT 1	1,000	const	PRIMARY	1	1	0	1
With PK	SELECT * FROM t WHERE id = N LIMIT 1	10,000	const	PRIMARY	1	1	0	1
With PK	SELECT * FROM t WHERE id = N LIMIT 1	100,000	const	PRIMARY	1	1	0	1
With PK	SELECT * FROM t WHERE id = N LIMIT 1	1,000,000	const	PRIMARY	1	1	0	1
Without PK	SELECT * FROM t WHERE id = N LIMIT 1	10	ALL	(none)	993,492	1	10	10
Without PK	SELECT * FROM t WHERE id = N LIMIT 1	1,000	ALL	(none)	993,492	1	1,000	1,000
Without PK	SELECT * FROM t WHERE id = N LIMIT 1	10,000	ALL	(none)	993,492	1	10,000	10,000
Without PK	SELECT * FROM t WHERE id = N LIMIT 1	100,000	ALL	(none)	993,492	1	100,000	100,000
Without PK	SELECT * FROM t WHERE id = N LIMIT 1	1,000,000	ALL	(none)	993,492	1	1,000,000	1,000,000

Rows actually read = Handler_read_rnd_next when MySQL scanned the table, otherwise Handler_read_key (index lookups). A scan also counts one final 'end of table' read, so a full scan of N rows shows N + 1.

Table size on disk (information_schema, after ANALYZE TABLE)

InnoDB stores the whole table inside its clustered index. With a PK that index is the PRIMARY KEY; without one InnoDB adds a hidden 6-byte row id (GEN_CLUST_INDEX) and clusters on that.

Run	Stage	Table	Row count	Data (MB)	Secondary indexes (MB)
1	after 1,000,000	With PK	1,000,000	83.6	0.0
1	after 1,000,000	Without PK	1,000,000	89.6	0.0
1	after 1,220,000	With PK	1,220,000	101.6	0.0
1	after 1,220,000	Without PK	1,220,000	109.6	0.0
2	after 1,000,000	With PK	1,000,000	83.6	0.0
2	after 1,000,000	Without PK	1,000,000	89.6	0.0
2	after 1,220,000	With PK	1,220,000	101.6	0.0
2	after 1,220,000	Without PK	1,220,000	109.6	0.0
3	after 1,000,000	With PK	1,000,000	83.6	0.0
3	after 1,000,000	Without PK	1,000,000	89.6	0.0
3	after 1,220,000	With PK	1,220,000	101.6	0.0
3	after 1,220,000	Without PK	1,220,000	109.6	0.0

Method & setup

Environment

Machine	MacBook (MySQL 8.0 in Docker)
CPU	Intel(R) Core(TM) i5-5257U CPU @ 2.70GHz
Operating system	Linux 6.10.14-linuxkit
MySQL version	8.0.46
Storage engine	InnoDB (MySQL default)
innodb_buffer_pool_size	128 MB (default)
innodb_flush_log_at_trx_commit	1 (default - full durability)
Client	Python 3.12.14 + mysql-connector-python, same machine as the server
Run started	2026-09-27 18:02:17 UTC

Procedure (from the assignment)

Step 1	Create two tables with identical columns: data_pk (id is the PRIMARY KEY) and data_nopk (id is a normal INT column: no primary key, no index).
Step 2	Insert 1,000,000 random rows (ids 1..1,000,000) into each table. Both tables get exactly the same random data.
Step 3	SELECT * FROM <table> WHERE id = N for N = 10, 1,000, 10,000, 100,000, 1,000,000; each query executed 5 times, median kept.
Step 4	Insert 20,000 new random rows into the same tables and time it.
Step 5	Insert 40,000, 60,000, 100,000 more rows (cumulative, same tables) and time each batch.
Repeats	The whole procedure (fresh tables) was repeated 3 times; the table order (PK first / no-PK first) alternates between runs so neither table always runs 'warm'. Summary values are medians of the runs.
Insert method	Multi-row INSERT statements of 1,000 rows each, one COMMIT per step. Random data generated before the timer starts, so only database time is measured.
Timing	Wall-clock time from sending the query to receiving the full result (Python time.perf_counter). The ~0.3 ms measured with a PK is mostly client round-trip overhead; the index lookup itself inside MySQL takes only microseconds.
Random data	name, email, age 18-65, one of 12 Indonesian cities, salary, created_at 2020-2025; seed 2026 + run number (reproducible).

Table definitions

With PK	<pre>CREATE TABLE data_pk (id INT NOT NULL, name VARCHAR(50) NOT NULL, email VARCHAR(100) NOT NULL, age INT NOT NULL, city VARCHAR(30) NOT NULL, salary DECIMAL(12,2) NOT NULL, created_at DATETIME NOT NULL, PRIMARY KEY (id)) ENGINE=InnoDB</pre>
Without PK	<pre>CREATE TABLE data_nopk (id INT NOT NULL, name VARCHAR(50) NOT NULL, email VARCHAR(100) NOT NULL, age INT NOT NULL, city VARCHAR(30) NOT NULL, salary DECIMAL(12,2) NOT NULL, created_at DATETIME NOT NULL) ENGINE=InnoDB</pre>

Why the results look like this

PRIMARY KEY	InnoDB stores the table as a B+tree ordered by the primary key (clustered index). Finding id = N walks ~3-4 tree levels no matter where the row is: $O(\log n)$, constant in practice.
-------------	---

No PRIMARY KEY	The id column has no index, so WHERE id = N is a full table scan: MySQL reads every row and compares id. Without LIMIT it cannot stop early (id is not known to be unique), so row 10 and row 1,000,000 cost the same: $O(n)$.
Hidden row id	A table without a PK is still clustered - InnoDB adds a hidden 6-byte DB_ROW_ID and builds the clustered index on it. That index is useless for WHERE id = N.
Inserts	With increasing ids, a PK insert appends to the right end of the B+tree (cheap). A random PK (e.g. UUID) would cause page splits and slower inserts; a table with extra secondary indexes would also insert more slowly.
Caveat	Absolute times depend on the machine (CPU, SSD, RAM, buffer pool). Compare the ratio between the two tables, not the raw milliseconds, when you rerun it on another laptop.